

Plataforma para gestão de serviços de apoio à comunidade

Liliana Cristina Fernandes da Conceição Paquete- n^a a43720
Verissimo Nancassa- n^a a41830

Trabalho realizado sob a orientação de

João Paulo Pereira

João Pereira

Licenciatura em Engenharia Informática

2023-2024

Plataforma para gestão de serviços de apoio à comunidade

Relatório da UC de Projeto
Licenciatura em Engenharia Informática
Escola Superior de Tecnologia e Gestão

Liliana Cristina Fernandes da Conceição Paquete- nº a43720
Verissimo Nancassa- nº a41830

2023-2024

A Escola Superior de Tecnologia e Gestão não se responsabiliza pelas opiniões expressas neste relatório.

Declaramos que o trabalho descrito neste relatório é da nossa autoria e é da nossa vontade que o mesmo seja submetido a avaliação.

Liliana Cristina Fernandes da Conceição Paquete- n^a a43720
Verissimo Nancassa- n^a a41830

Liliana Cristina Fernandes da Conceição Paquete- n^a a43720
Verissimo Nancassa- n^a a41830

Agradecimentos

Primeiramente agradecemos a Deus por ter nos guiado e protegido até aqui.

Este projeto não teria sido realizado da mesma forma sem a colaboração e a opinião de algumas pessoas, às quais nos compete agradecer.

Agradecemos ao nosso orientador, professor João Paulo Pereira, pela disponibilidade e apoios oferecidos ao longo deste projeto.

O nosso coorientador, o professor João Pereira, pela disponibilidade e recomendações dadas ao longo do projeto.

Às nossas famílias por todos os apoios e incentivos dados durante a realização deste projeto, ajudando a que o mesmo se tivesse concretizado.

Resumo

Este relatório apresenta o desenvolvimento de uma plataforma para gestão e requisição dos serviços de apoio à comunidade oferecidos pela ESTIG. O principal objetivo deste projeto é desenvolver uma plataforma web para a gestão e requisição eficiente dos serviços prestados pela ESTIG, automatizando e digitalizando todo o processo, dos laboratórios especializados da ESTIG, que oferecem serviços nas áreas de Química, Construção Civil, Informática e Biologia. A motivação para este projeto reside na necessidade de melhorar a eficiência operacional, reduzir as falhas humanas e a perda de informação, e proporcionar uma experiência mais satisfatória tanto para os clientes como para os gestores dos serviços.

A metodologia utilizada inclui a análise dos processos atuais, o levantamento de requisitos, o desenvolvimento da plataforma utilizando C#, Visual Studio, SQL Workbench, Astah UML e SQLite, e a implementação de um sistema de gestão completo, desde o pedido inicial até à conclusão.

As conclusões evidenciam a importância de uma solução integrada para a gestão dos serviços de apoio à comunidade e demonstram a relevância de ferramentas que promovam a eficiência e organização no sector académico.

Palavras-chave: C#, UWP, Software de gestão de serviços.

Abstract

This report presents the development of a platform for managing the community support services offered by ESTIG. The main objective of this project is to develop a web platform for the efficient management of the services provided by ESTIG, automating and digitising the entire process of ESTIG's specialised laboratories, which offer services in the areas of Chemistry, Construction, Computer Science and Biology. The motivation for this work lies in the need to improve operational efficiency, reduce human error and loss of information, and provide a more satisfactory experience for both customers and service managers.

The methodology used includes analysing current processes, gathering requirements, developing the platform using C#, Visual Studio, SQL Workbench, Astah UML and SQLite, and implementing a complete management system, from the initial request to completion and invoicing.

The conclusions highlight the importance of an integrated solution for managing community support services and demonstrate the relevance of tools that promote efficiency and organisation in the academic sector.

Keywords: C#, UWP, Service management software.

Conteúdo

Agradecimentos	5
Resumo	6
Abstract	7
Conteúdo	8
Lista de Figuras	10
Siglas	11
Introdução	12
1.1 Enquadramento.....	12
1.2 Objetivos	13
1.3 Estrutura do documento	14
Contexto e Tecnologias/Ferramentas	15
2.1 C#	15
2.1.1 O que é?	15
2.1.2 A .NET (Network Enabled Technology)	16
2.1.3 Vantagens.....	16
2.2 Visual Studio	16
2.2.1 Vantagens.....	17
2.3 Interface de Utilizador.....	17
2.3.1 A Plataforma Universal do Windows (UWP).....	18
2.3.2 Vantagens.....	18
2.4 Base de Dados	19
2.4.1 SQLite.....	19
2.4.2 Vantagens.....	19
2.4.3 MySQL Workbench.....	20
2.5 Astah UML.....	20
2.5.1 Vantagens.....	21
Abordagem/Análise/Modelação	22
3.1 Modelação	23
3.1.1 Atores.....	23

3.2	Requisitos	24
3.3	Diagrama Casos de Uso	29
3.4	Diagrama ER	29
3.5	Mockups	30
Desenvolvimento/Implementação		31
4.1	Arquitetura da plataforma	31
4.1.1	Arquitetura das camadas	31
4.1.2	Camada de Domínio	33
4.1.3	Camada de infraestrutura	34
4.1.3	Camada de apresentação	34
4.2	DbContext	Erro! Marcador não definido.
4.2.1	Criação das Tabelas	35
4.2.2	Relação entre as tabelas	38
Interface.....		48
5.1	Área do Utilizador	52
5.1.1	Admin	52
5.1.2	Cliente	55
5.1.3	Responsável de Laboratório.....	56
5.1.4	Técnico Laboratório.....	57
Conclusões		59
Bibliografia.....		1
Apêndice A.		1
Proposta Original do Projeto.....		Erro! Marcador não definido.
A <Proposta de Projeto>.....		Erro! Marcador não definido.
A.1	Erro! Autorreferência de marcador inválida.	Erro! Marcador não definido.
Apêndice B.		Erro! Marcador não definido.
Outro(s) Apêndice(s)		Erro! Marcador não definido.
B <Nome do Anexo>		Erro! Marcador não definido.
B.1	Erro! Autorreferência de marcador inválida.	Erro! Marcador não definido.

Lista de Figuras

Figura 1 -Diagrama de caso de uso	29
Figura 2-Diagrama ER	30
Figura 3- Arquitetura.....	32
Figura 4- Organização do projeto.....	33
Figura 5- página inicial	48
Figura 6- Registo.....	50
Figura 7- Login	51
Figura 8- Recuperação de Senha.....	51
Figura 9- Confirmação da Nova senha.....	52
Figura 100 - Serviços	49
Figura 11- Laboratórios.....	53
Figura 12 - Pedidos	Erro! Marcador não definido.
Figura 13 -Tipo de Utilizadores (Clientes).....	54
Figura 14 -Tipo de Utilizadores (Técnicos e Responsáveis de Laboratórios).....	55
Figura 15 -Página Inicial Cliente	56
Figura 16 - Meus Pedidos (Cliente)	56
Figura 17 - Página Técnico de Laboratório (Elaboração de orçamento).....	57
Figura 18 - Pedidos (Técnico de Laboratório)	58

Siglas

ESTiG Escola Superior de Tecnologia e Gestão.

.NET Network Enabled Technology

IoT Internet das Coisas

UWP Universal Windows Platform

API Application Programming Interface

SQL Structured Query Language

UML Unified Modeling Language

Introdução

Com o objetivo de otimizar a gestão e a requisição dos serviços oferecidos pelos laboratórios da ESTIG, desenvolvemos uma plataforma web que automatiza e digitaliza todo o processo de solicitação e gestão dos serviços.

A plataforma foi projetada para ser intuitiva e eficiente, proporcionando uma experiência de utilização fluida e adaptada às necessidades dos utilizadores.

Para garantir que a plataforma responde às necessidades específicas dos serviços prestados pela ESTIG, foi essencial efetuar uma análise detalhada dos processos existentes.

Esta análise incluiu um conhecimento profundo dos fluxos de trabalho, das interações com os clientes e das necessidades de gestão interna, permitindo-nos desenvolver uma solução que não só simplifica os processos, como também melhora a qualidade do serviço prestado. Além disso, foi dada especial atenção à arquitetura do sistema, garantindo que a plataforma é segura.

Este projeto cobre a análise de requisitos, o design, a implementação, os testes e a implantação da solução, utilizando tecnologias como C#, Visual Studio, SQL Workbench, Astah UML e SQLite.

1.1 Enquadramento

A ESTIG (Escola Superior De Tecnologia e Gestão) oferece diversos serviços em diversas áreas da ciência.

No entanto, o processo de gestão destes serviços, incluindo a requisição e administração dos serviços, tem sido tradicionalmente realizado de forma manual, recorrendo a documentos em papel e procedimentos que, embora funcionais, são suscetíveis a falhas humanas.

A ausência de uma ferramenta digital no mercado que centralize e automatize a gestão desses serviços representa uma lacuna significativa no sistema atual, tornando o processo suscetível

a falhas como extravio de documentos, atrasos na comunicação e dificuldades na localização de pedidos. Essas falhas não só comprometem a eficiência dos serviços prestados pela ESTIG, como também afetam a satisfação dos clientes.

Perante este cenário, foi decidido desenvolver uma plataforma digital inovadora, concebida especificamente para otimizar a gestão destes serviços. A plataforma tem como objetivo automatizar o fluxo de trabalho desde o pedido inicial até à conclusão do serviço, garantindo maior precisão, rapidez e transparência em todas as fases do processo.

Com a implementação desta solução, espera-se não só minimizar as falhas humanas, mas também promover um ambiente mais sustentável e tecnologicamente avançado, onde a inovação e a eficiência são pilares fundamentais na prestação dos serviços.

1.2 Objetivos

O principal objetivo deste projeto é desenvolver uma plataforma web para a gestão eficiente dos serviços para e que os clientes possam fazer requisições dos serviços prestados pela ESTIG de uma forma mais objetiva e fácil, automatizando e digitalizando todo o processo, desde o pedido inicial até à conclusão.

Para atingir este objetivo, foram definidos os seguintes passos:

- Analisar as necessidades e desafios dos atuais processos manuais na gestão dos serviços prestados pelos laboratórios da ESTIG para melhorar a eficiência e reduzir as falhas.
- Desenvolver uma arquitetura de sistema robusta e escalável que possa integrar os vários serviços oferecidos pela ESTIG instituição, garantindo segurança, facilidade de utilização e adaptação futura.
- Criar uma interface intuitiva e acessível para os utilizadores, que simplifique a submissão e o acompanhamento dos pedidos.
- Implementar funcionalidades que garantam o controlo de qualidade e acompanhamento dos serviços.

- Implementar um sistema de notificações que mantenha os utilizadores informados sobre as suas requisições e qualquer atualização relevante.

Com a concretização destes passos, espera-se criar uma plataforma que melhore a eficiência operacional destes serviços prestados pelos laboratórios da ESTIG.

1.3 Estrutura do documento

1. No Capítulo 1, é feita uma introdução e um enquadramento do projeto a realizar e são apresentados os objetivos que se pretende concretizar.
2. No Capítulo 2, é feita uma abordagem à parte teórica das tecnologias utilizadas.
3. No Capítulo 3, é feita uma apresentação do diagrama Casos de Uso e do modelo Entidades-Relações que foram implementados durante a realização do projeto.
4. No Capítulo 4, é feita uma descrição do desenvolvimento e implementação do projeto, é apresentado e explicado os códigos mais importantes.
5. No Capítulo 5, é apresentado todo interface desenvolvido, teste, avaliação apresentando imagens e explicações de cada página que constituem o projeto.
6. Por último, no Capítulo 6, são apresentadas as conclusões retiradas do projeto desenvolvido.

Contexto e Tecnologias/Ferramentas

O objetivo deste capítulo é apresentar e explicar o modelo de gestão implementado no projeto. Serão também detalhadas as tecnologias, linguagens de programação e ferramentas utilizadas na realização do projeto, justificando as escolhas feitas.

A plataforma proposta para a gestão dos serviços foi desenvolvida utilizando um conjunto específico de tecnologias e ferramentas, selecionadas para garantir eficácia, segurança e facilidade de utilização.

A plataforma foi desenvolvida utilizando a linguagem de programação C# no ambiente de desenvolvimento integrado (IDE) Visual Studio. Esta combinação oferece um conjunto robusto de ferramentas para desenvolver, depurar e manter o software, garantindo um ciclo de desenvolvimento eficiente e de alta qualidade.

2.1 C#

1.1.1 O que é?

é uma linguagem de programação moderna, orientada para objectos e fortemente tipada, desenvolvida pela Microsoft como parte da sua plataforma .NET. Projetada para ser simples, eficiente e segura, o C# combina o poder de linguagens como o C++ com a simplicidade e a facilidade de utilização de linguagens como o Java. [1]

1.1.2 A .NET (Network Enabled Technology)

é uma plataforma de desenvolvimento gratuita e de código aberto que pode ser utilizada em vários sistemas operativos, como o Windows, o Linux e o macOS. Esta estrutura permite criar vários tipos de aplicações, incluindo aplicações Web, aplicações de ambiente de trabalho, aplicações móveis, jogos e aplicações para a Internet das Coisas (IoT). [2]

1.1.3 Vantagens

A escolha da linguagem de programação C# para o desenvolvimento do projeto baseou-se em vários factores-chave.

O C# é uma linguagem robusta e escalável, ideal para a criação de aplicações seguras e de grande escala. A integração com o .NET Framework oferece uma vasta gama de bibliotecas e ferramentas que facilitam o desenvolvimento de funcionalidades avançadas, como segurança, acesso a dados e comunicação em rede. A sua utilização em conjunto com o Visual Studio permite um desenvolvimento rápido e produtivo, graças às ferramentas de depuração avançadas e ao suporte para sistemas de controlo de versões.

O C# é também conhecido pelas suas características de segurança e gestão de memória, garantindo a estabilidade das aplicações. Além disso, a ampla adoção da linguagem e o forte apoio da comunidade oferecem uma abundância de recursos, facilitando a resolução de problemas e a aprendizagem. A compatibilidade com tecnologias de bases de dados como o SQL Server foi outro fator importante, permitindo uma gestão eficiente dos dados. Em suma, o C# foi escolhido pela sua eficiência, segurança e facilidade de manutenção e expansão.[3]

2.2 Visual Studio

é um ambiente de desenvolvimento integrado (IDE) da Microsoft, amplamente utilizado para desenvolver uma variedade de aplicações, incluindo software de ambiente de trabalho, aplicações Web, serviços na nuvem, jogos e muito mais. Desde o seu lançamento inicial em 1997, o Visual Studio evoluiu significativamente, incorporando uma vasta gama de funcionalidades que o tornam uma ferramenta poderosa e versátil para os programadores.[4]

1.1.4 Vantagens

O Visual Studio foi escolhido para o desenvolvimento do projeto devido à sua integração nativa com o C# e a plataforma .NET, oferecendo uma experiência de desenvolvimento fluida e eficiente. O IDE disponibiliza ferramentas avançadas de debugging, essenciais para a rápida identificação e correção de erros, além de incluir funcionalidades que aumentam a produtividade, como a sugestão e reestruturação automática de código.

O Visual Studio também suporta uma vasta gama de testes, garantindo a robustez e a qualidade da aplicação. A sua extensibilidade permite que o ambiente de trabalho seja personalizado através de extensões, e a extensa documentação, juntamente com uma comunidade ativa, oferece um apoio valioso aos programadores.

O Visual Studio é um dos IDE mais populares para o desenvolvimento .NET, com uma vasta comunidade de programadores que partilham conhecimentos, tutoriais, plugins e soluções para problemas comuns, o que é extremamente útil tanto para programadores principiantes como para programadores experientes.

Estas características fazem do Visual Studio a escolha ideal para projectos complexos e de grande escala.[5]

2.3 Interface de Utilizador

A plataforma foi projetada utilizando UWP para criar uma interface de utilizador intuitiva e reativa, permitindo que a aplicação funcione de forma consistente em vários dispositivos Windows 10. A UWP facilita o desenvolvimento de interfaces modernas e interativas que melhoram a experiência do utilizador.

1.1.5 A Plataforma Universal do Windows (UWP)

é um conjunto de APIs da Microsoft para o desenvolvimento de aplicações, introduzido com o Windows 10. O seu objetivo é facilitar a criação de aplicações de interface modernas que possam ser executadas no Windows 10 e no Windows 10 Mobile, sem a necessidade de escrever código específico para cada sistema operativo. A UWP suporta o desenvolvimento em linguagens como C++, C#, XAML e VB.NET, e a API subjacente é escrita em C#, com suporte para C++, VB.NET, C#, F# e JavaScript. Como uma extensão da API de tempo de execução do Windows, introduzida no Windows 8, a UWP permite-lhe criar aplicações compatíveis com qualquer sistema que suporte esta API, conhecida como Aplicações Universais.[6]

1.1.6 Vantagens

A escolha do UWP para o desenvolvimento da interface do projeto baseia-se nas suas vantagens.

As aplicações UWP são seguras e declaram os dados e recursos do dispositivo a que acedem, exigindo autorização do utilizador. Podem utilizar uma API comum em todos os dispositivos Windows, adaptar a interface a diferentes dispositivos e tamanhos de ecrã e estão disponíveis na Microsoft Store para Windows 10 e 11, oferecendo várias formas de monetização.

Para além disso, podem ser instaladas e desinstaladas sem risco de causar danos no computador. Os aplicativos UWP também podem despertar interesse através de recursos como blocos dinâmicos, notificações push e integração com recursos do sistema, como a Linha do Tempo do Windows.

Podem ser programadas em C#, C++, Visual Basic e JavaScript, com interfaces de utilizador desenvolvidas em WinUI, XAML, HTML ou DirectX.[7]

2.4 Base de Dados

O SQLite é escolhido a ser utilizado para o armazenamento e administração eficiente dos dados. Este sistema de administração de bases de dados relacionais proporciona um elevado desempenho, segurança e escalabilidade, garantindo que os dados e serviços dos utilizadores são geridos de forma eficiente e segura.

Uma ferramenta utilizada para a modelação de bases de dados, que permite a criação e visualização intuitiva de modelos de dados complexos. A utilização do Workbench facilita a organização e estruturação dos dados, garantindo uma base sólida para o desenvolvimento da plataforma.

1.1.7 SQLite

É uma biblioteca compacta que implementa uma base de dados SQL sem a necessidade de um servidor. É considerada uma biblioteca portátil, utilizando a SQL (Structured Query Language) como linguagem padrão para bases de dados relacionais. Sendo de código aberto, pode ser utilizada para fins pessoais, académicos ou profissionais em várias plataformas.[8]

1.1.8 Vantagens

Utilizamos o SQLite devido à sua elevada compatibilidade com vários sistemas operativos, como Windows, OS, Linux, Android e iOS, garantindo um desempenho consistente.

Além disso, suporta várias linguagens de programação, simplificando o desenvolvimento de aplicações, uma vez que não requer qualquer configuração ou relação utilizador-servidor. Sendo de domínio público, pode ser utilizado livremente para qualquer finalidade, o que aumenta a sua escolha.

O SQLite é autónomo, não necessitando de bibliotecas externas ou de configurações complexas, o que o torna ideal para aplicações embebidas, como smartphones e consolas. As suas tabelas dinâmicas permitem flexibilidade na inserção de dados, e as ligações únicas facilitam a gestão e a transferência de bases de dados. .[8]

1.1.9 MySQL Workbench

é uma ferramenta visual integrada que facilita o desenvolvimento, a concepção e a administração de bases de dados MySQL. Desenvolvida pela Oracle Corporation, esta ferramenta é amplamente utilizada por administradores de bases de dados (DBAs), programadores e arquitectos para gerir e modelar bases de dados MySQL de forma eficiente e intuitiva. [9]

2.4.1.1 Vantagens

O SQL Workbench foi escolhido devido às suas vantagens:

O MySQL Workbench é uma ferramenta abrangente e intuitiva para gerir bases de dados MySQL, abrangendo três áreas principais: modelação de dados, desenvolvimento de SQL e administração de bases de dados.

Na modelação de dados, permite-lhe criar e modificar visualmente esquemas de bases de dados, facilitando a criação de tabelas, a definição de relações entre tabelas e a adição de índices e chaves externas.

Para o desenvolvimento de SQL, o Workbench oferece um ambiente avançado de edição de código com características como realce de sintaxe, preenchimento automático e verificação de erros, ajudando-o a escrever e executar consultas SQL de forma eficiente e precisa.

Na administração de bases de dados, o MySQL Workbench permite-lhe gerir utilizadores e permissões, monitorizar o desempenho e efetuar cópias de segurança e restauros, facilitando a manutenção e a segurança dos dados. Além disso, o Workbench integra-se perfeitamente com outras ferramentas da família MySQL, como o MySQL Server, o MySQL Router e o MySQL Shell, proporcionando uma experiência unificada e eficiente para a administração de bases de dados MySQL.[9]

2.5 Astah UML

é uma ferramenta CASE (Computer-Aided Software Engineering) amplamente utilizada para modelar soluções de software com UML. Disponível em versões gratuitas

“comunitárias” e pagas “profissionais”, o Astah é desenvolvido em Java e facilita a criação de modelos que reflectem o pensamento humano. A ferramenta permite transformar modelos em código, através de engenharia direta, e também converter código existente em modelos UML, utilizando engenharia inversa.[9]

1.1.10 Vantagens

Escolhemos o Astah UML devido às suas vantagens e ao facto de ser uma ferramenta intuitiva.

O Astah é amplamente reconhecido e utilizado no mercado para a modelação de software utilizando UML e modelação ER. Destaca-se pela disponibilidade de uma versão gratuita que atrai muitos utilizadores e contribui para o seu desenvolvimento contínuo. Através da comunidade de utilizadores Astah e do Astah SDK, os utilizadores podem trocar informações e criar plugins personalizados.

A versão profissional da ferramenta oferece suporte completo para todos os diagramas UML e modelação ER, bem como funcionalidades avançadas como a criação de diagramas de mapas mentais e a geração de código em Java, C#, C++ e PHP, com base em diagramas de classes ou ER. O Astah também permite a engenharia inversa, convertendo código existente em diagramas UML.

Reconhecida pela sua singularidade, a ferramenta inclui funcionalidades para exportação de diagramas e rastreio de requisitos, tornando-a ideal para engenheiros de software e analistas de sistemas. A versão gratuita, embora limitada em algumas características, serve como uma excelente introdução à ferramenta, enquanto a versão paga oferece funcionalidades avançadas.[10]

Abordagem/Análise/Modelação

Neste capítulo serão apresentados, o diagrama Casos de Uso, modelo Entidade Relacional, requisitos e os atores que foram implementados durante a realização do projeto. Esta etapa ou fase é muito importante pois antes da parte do desenvolvimento projeto é essencial realizar o levantamento de requisitos, escolher as ferramentas necessárias para a implementação da plataforma bem como realizar a sua modelação pois estabelece a base sobre a qual será desenvolvida a plataforma, ou seja como e o que será desenvolvida na plataforma, o que cada utilizador pode, deve ou não fazer dentro do sistema, e como o sistema da plataforma pode ou deve operar cada uma das suas funções mediante varias questões como desempenho, segurança aspetos de usabilidade, etc... sem deixa de salientar que esta fase garante que a equipe de desenvolvimento compreenda claramente as necessidades e expectativas dos usuários finais (clientes), evitando assim a má interpretação do que foi pedido o que pode retardar e conclusão do projeto, e garantido um aumento considerável na satisfação do usuário final. Dito isso neste capítulo serão apresentados tais processos da seguinte forma: no índice 3.1 a modelação que nesse caso serão apresentados os atores, não secção 3.2 são apresentados os requisitos funcionais e não funcionais da plataforma, 3.3 é apresentado o diagrama de casos de uso, no 3.4 o diagrama ER(Entidade relacional) e por fim 3.5 são apresentadas as Mockups iniciais da plataforma.

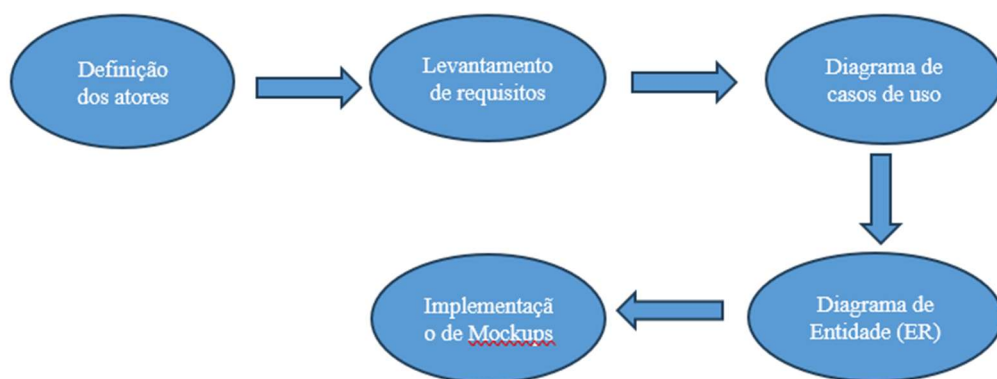


Figura 1: Etapas do planeamento do projecto.

3.1 Modelação

1.1.11 Atores

Antes de mais precisamos saber quem ou quais serão os possíveis utilizadores da plataforma para que as funcionalidades dela corresponda e satisfaça as suas necessidades dos mesmos, dito isso para a implementação desta plataforma foram levantados 4 tipos de utilizadores que são:

- Responsável do gabinete (Administrador)
- Responsável do laboratório
- Cliente
- Técnico de Laboratório

Responsável do gabinete (Admin):

Este ator é o responsável pela administração da plataforma, ele/a tem o poder de inserir novos utilizadores via link de convite e ou também efetuar o registo do cliente caso o cliente não tenha a disponibilidade de o fazer, também faz a administração dos pedidos, os mandando para os responsáveis do laboratório, gerência os serviços na plataforma como também os pedidos.

Responsável do laboratório:

Ele é o responsável pela inserção do/s técnico/s de seu determinado laboratório na plataforma, também determina se o pedido feito pelo cliente mediante a disponibilidade do laboratório poder ser feito ou não e faz a validação do orçamento do serviço(s) solicitado(s) para depois ser entregue responsável de gabinete(Admin).

Técnico do laboratório:

Este é o responsável pela elaboração do orçamento do pedido feito pelo cliente depois de aceite pela instituição, orçamento este que passará pela aprovação dos outros utilizadores.

Cliente:

Este é o utilizador principal da aplicação, ele é capaz de efetuar registo na plataforma, efetuar o login, efetuar os pedidos, aceitar ou não o orçamento como também acompanhar o estado do seu pedido.

3.2 Requisitos

Aqui irá ser descrito cada requisito de sistema que será posteriormente mapeado para o diagrama de caso de uso apresentado posteriormente no capítulo 3.3, os requisitos funcionais serão agrupados mediante o tipo de utilizador.

3.2.1 3.2.1 Requisitos Funcionais

3.2.2 Cliente

- Efetuar registo na Plataforma;
Descrição: O cliente deve criar uma conta na plataforma para poder utilizar os serviços oferecidos.
- Efetuar Login na Plataforma;
Descrição: Após o registo, o cliente deve ser capaz de aceder à plataforma, introduzindo o email e a password fornecidos durante o registo.
- Efetuar Logout na Plataforma;
Descrição: O cliente deve poder sair da sua conta na plataforma de forma segura, terminando a sessão ativa.
- Requisitar do serviço;
Descrição: O cliente pode solicitar um serviço específico oferecido pela plataforma
- Editar seus dados na aplicação, morada, email, telefone, password;
Descrição: O cliente pode atualizar os seus dados pessoais, como morada, email, telefone e password, diretamente na plataforma.
- Consultar seu histórico de Pedidos;

Descrição: O cliente tem acesso a um histórico de todos os serviços solicitados anteriormente, incluindo detalhes como datas, descrições de serviços e o estado atual de cada pedido.

- Aceitar orçamento do pedido;

Descrição: Depois de solicitar um serviço, o cliente recebe um orçamento. O cliente pode aceitar este orçamento, o que dá início ao processo de execução do serviço

- Recusar Orçamento proposto pelo serviço;

Descrição: Se o cliente não estiver satisfeito com o orçamento recebido, tem a possibilidade de o recusar.

- Solicitar um novo orçamento;

Descrição: Após rejeitar um orçamento, o cliente pode solicitar um novo orçamento, quer ajustando os requisitos do serviço, quer pedindo uma revisão ao prestador de serviços.

- Visualizar o estado do seu pedido;

Descrição: O cliente pode acompanhar a evolução do serviço, com estados como “Pendente”, “Em curso”, “Concluído”, etc.

- Avaliar serviço prestado

Descrição: Após a conclusão do serviço, o cliente pode avaliar a qualidade do serviço recebido, deixando comentários ou classificações.

Administrador

- Efetuar registo na Plataforma;

Descrição: o administrador cria uma conta para si ou para outros utilizadores na plataforma, definindo as permissões adequadas.

- Efetuar Login na Plataforma;

Descrição: O administrador acede à plataforma introduzindo as suas credenciais de acesso.

- Efetuar Logout na Plataforma;

Descrição: O administrador encerra a sessão na plataforma de forma segura.

- Adiciona Serviços ao catálogo;

Descrição: O administrador tem a capacidade de adicionar novos serviços ao catálogo disponível para os clientes na plataforma.

- Arquivar serviço;

Descrição: O administrador pode remover ou arquivar serviços que não são mais oferecidos.

- Atualizar serviço;

Descrição: O administrador pode modificar as descrições, preços ou outros detalhes dos serviços existentes.

- Receber notificações sobre pedidos solicitados;

Descrição: O administrador é notificado sempre que um cliente solicita um novo serviço.

- Validar pedido;

Descrição: O administrador revê e aprova os pedidos recebidos dos clientes antes de proceder à sua execução.

- Recusar pedido caso o cliente esteja com a situação irregular;

Descrição: O administrador pode rejeitar pedidos de clientes que não atendam aos requisitos, como pagamentos atrasados ou não cumprimento das regras.

- Enviar pedido ao responsável de laboratório;

Descrição: depois de validado, o administrador encaminha o pedido para o responsável adequado.

- Analisar orçamento enviado pelo responsável do laboratório;

Descrição: O administrador analisa o orçamento proposto pelo responsável de laboratório para o serviço solicitado.

- Pedir reavaliação com comentários sobre o orçamento ao responsável de laboratório;

Descrição: se necessário, o administrador pode solicitar ajustes ou esclarecimentos sobre o orçamento fornecido.

- Enviar emails de atualização do pedido ao cliente;

Descrição: o administrador pode enviar atualizações ao cliente sobre o estado da sua encomenda.

- Inserir a Margem de preço do serviço ao orçamento;

Descrição: O administrador adiciona a margem de lucro ao orçamento final antes de o enviar ao cliente.

- Arquivar Pedido solicitado;

Descrição: O administrador pode arquivar pedidos concluídos ou cancelados, mantendo o sistema organizado.

- Atualizar Pedido solicitado;

Descrição: O administrador pode modificar os detalhes da encomenda, tais como o estado, as notas ou o prazo de entrega.

- Arquivar Utilizador na Plataforma;

Descrição: O administrador pode arquivar utilizadores que já não estão ativos na plataforma, como antigos clientes ou funcionários.

- Enviar Links de registos aos futuros utilizadores da plataforma, tantos clientes como responsáveis de laboratório;

Descrição: O administrador pode enviar convites a novos utilizadores para se registarem na plataforma.

- Inserir novo utilizador a plataforma;

Descrição: O administrador pode adicionar manualmente novos utilizadores à plataforma, atribuindo-lhes as permissões adequadas.

- Efetuar upload de Documentos Excel;

Descrição: O administrador pode carregar na plataforma documentos em formato Excel.

- Atualizar o estado do pedido;

O administrador pode alterar o estado do pedido.

Responsável de Laboratório

- Efetuar Login na Plataforma;

Descrição: O responsável de laboratório acede à plataforma utilizando as suas credenciais previamente configurado pelo administrador.

- Efetuar Logout na Plataforma;

Descrição: O responsável de laboratório termina a sessão na plataforma de forma segura.

- Inserir Técnicos do laboratório na plataforma;

Descrição: O responsável de laboratório pode adicionar técnicos à plataforma, permitindo-lhes participar na execução de encomendas.

- Inserir Orçamento do Pedido;

Descrição: o responsável de laboratório insere o orçamento que é previamente feito pelo técnico.

- Alterar Orçamento do Pedido;

Descrição: Se necessário, o responsável de laboratório pode ajustar o orçamento

- Validar orçamento;

Descrição: O responsável de laboratório confirma e valida o orçamento antes de o enviar ao administrador

- Efetuar upload de Documentos Excel;

Descrição: O responsável de laboratório pode carregar documentos em formato Excel, tais como listas de materiais.

- Atualizar o estado do pedido;

Descrição: O responsável de laboratório pode alterar o estado de uma encomenda.

- Emitir alertas de atualização do pedido;

Descrição: o responsável de laboratório pode alterar o estado dos pedidos.

Técnico

- Efetuar Login na Plataforma;

Descrição: O técnico acede à plataforma utilizando as suas credenciais de login fornecidas pelo responsável de laboratório.

- Efetuar Logout na Plataforma;

Descrição: O técnico termina a sessão na plataforma de forma segura.

- Elaborar o orçamento;

Descrição: O técnico efetua os cálculos necessários para determinar o custo de um serviço solicitado

- Elaborar a documentação do pedido;

Descrição: O técnico prepara toda a documentação necessária associada à encomenda.

- Enviar orçamento ao Responsável do Laboratório;

Descrição: Após a preparação, o técnico submete o orçamento ao responsável do laboratório para revisão e validação.

3.2.3 3.2.1 Requisitos não Funcionais

- Metodologia: Deve ser utilizado a mais ágil para o desenvolvimento da plataforma.
- Linguagem de programação: Foi utilizado a linguagem c#

- Banco de dados: Foi utilizado o Sqlite.
- Interface gráfica: a plataforma deve ser fácil de usar, objetiva e intuitiva.

3.3 Diagrama Casos de Uso

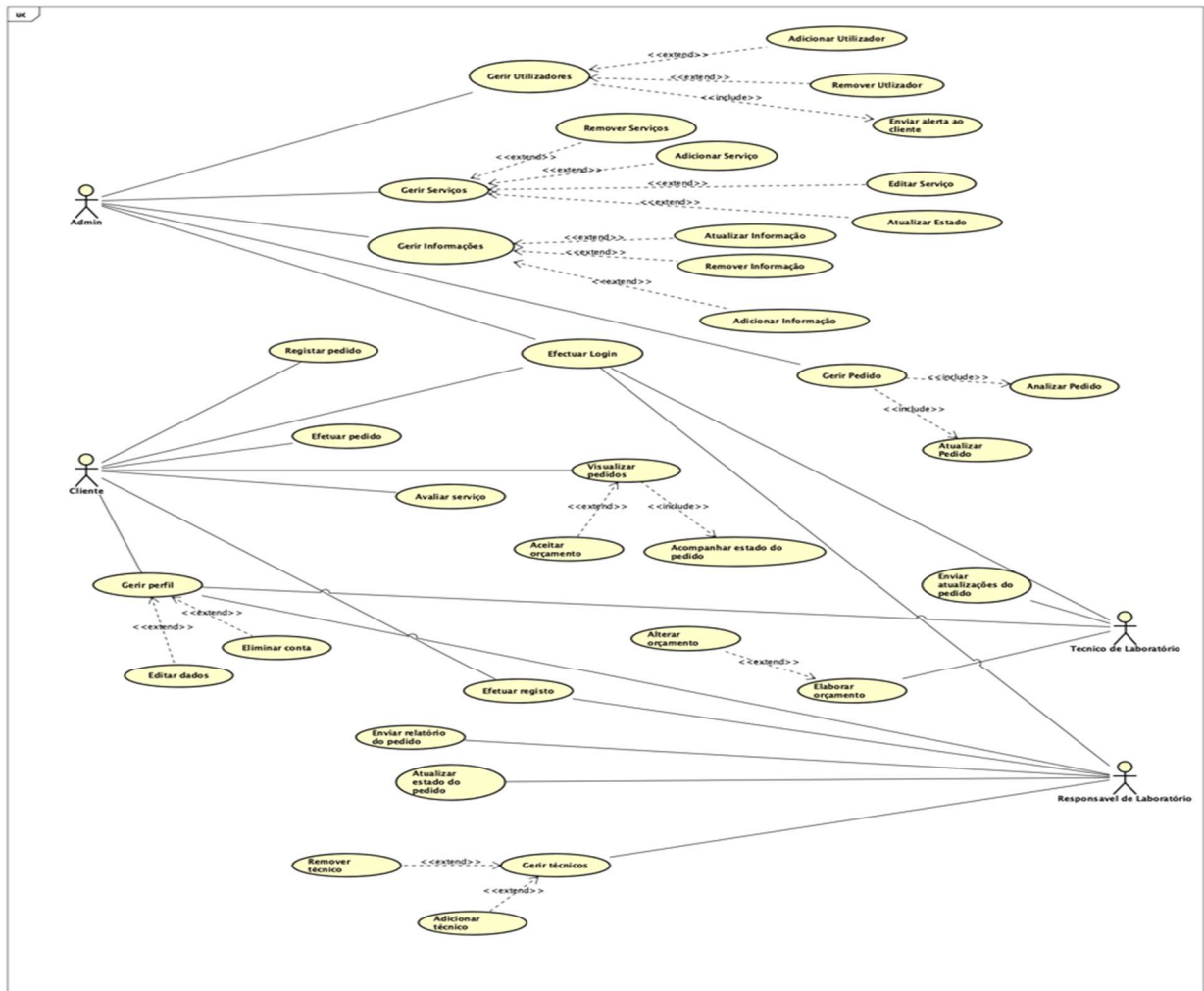


Figura 2 -Diagrama de caso de uso

3.4 Diagrama ER

Aqui dá-se a implementação da modelagem de dados, desta forma se saberá como os estes serão estruturados e armazenados na base de dados. A figura abaixo demonstra a elaboração desta etapa.

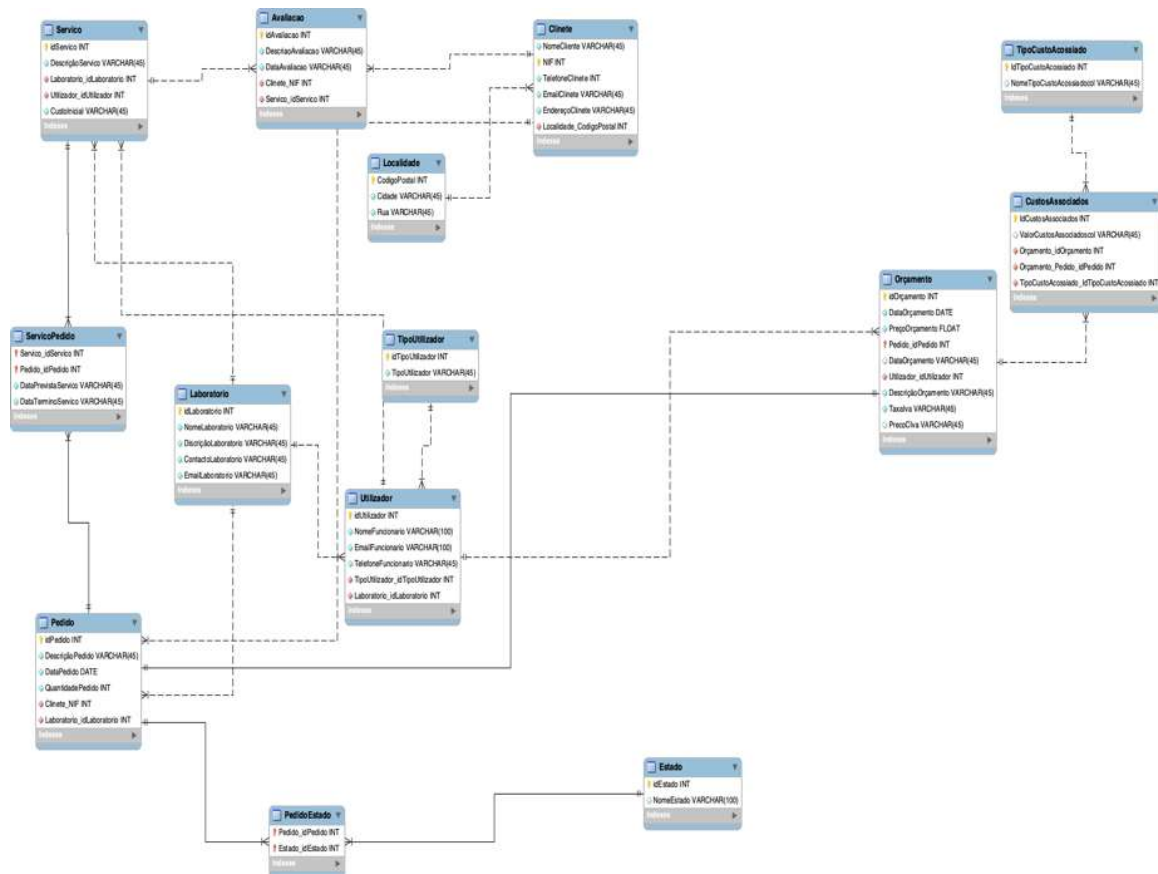


Figura 3-Diagrama ER

3.5 Mockups

Nota:

Por se tratar de um ficheiro extenso, em anexo, seguem os mockups iniciais da aplicação. Estes foram elaborados com base no requisitos definidos inicialmente e visam ilustrar as funcionalidades e o layout da plataforma, vale ressaltar que os mesmos poderão sofrer ajustes ou alterações no decorrer da fase de desenvolvimento da plataforma, tais modificações deverão ser recebidas pelos clientes, como também de forma a lidar e ou resolver alguns problemas que foram aparecendo, na fase de Desenvolvimento (capítulo 4).

4

Desenvolvimento/Implementação

Este capítulo visa apresentar e explicar o processo de desenvolvimento da aplicação apresentada anteriormente nos capítulos posteriores. Aqui será feita a descrição do desenvolvimento do projeto bem como quais as instalações e/ou configurações foram necessárias para o seu funcionamento. O capítulo está organizado a seguinte forma: no Índice 4.1 será apresentada a arquitetura do projeto. No 4.2 é apresentada a descrição da implementação da base de dados e operações de gestão da mesma. No 4.3 é descrito a implementação da plataforma bem como dos requisitos apresentados no capítulo 3, e por fim no 4.4 será apresentado as considerações finais mediante este capítulo como por exemplo os objetivos alcançados, os problemas e dificuldades no decorrer da implementação do projeto

4.1 Arquitetura da plataforma

Para a realização deste projeto, foi usada uma arquitetura em camadas, sendo estas camadas, de apresentação (Presentation Layer), a camada de lógica de negócio (Business Logic Layer) e a camada de dados.

4.2 Arquitetura das camadas

Para criar esta aplicação foram criados 3 projetos diferentes, cada um com a sua funcionalidade ou responsabilidade na Plataforma, o que facilita a manutenção e o

desenvolvimento, uma vez que a arquitetura em camadas consiste em dividir a aplicação em módulos distintos, cada um com a sua funcionalidade específica e que estabelecem comunicações entre si. Esta separação permite que cada camada desenvolvida seja testada e mantida de forma independente, o que promove a modularidade e a reutilização de código, o que ajuda na manutenção e evolução do sistema.

A maioria das arquiteturas em camadas é constituída por 4 camadas normalizadas: apresentação do negócio, persistência e base de dados. As camadas de negócio e de persistência ou as camadas de dados e de persistência podem ser combinadas numa única camada, o que pode acontecer se os elementos da camada de persistência forem utilizados nos componentes da camada de negócio.

Como referido anteriormente, neste cenário optou-se por apenas 3 camadas: a camada de Domínio (ProjectoFinal08.Domain) que define a lógica de negócio e os Modelos da aplicação, a camada de Infraestrutura (ProjectoFinal08.Infrastructure) onde é implementada a lógica de persistência e acesso aos dados, e a camada de Apresentação (ProjectoFinal08.UWP) que não é mais do que a implementação da interface com o utilizador.

A figura abaixo ilustra a estrutura da arquitetura utilizada e o propósito e funcionamento de cada uma delas será detalhado a seguir:

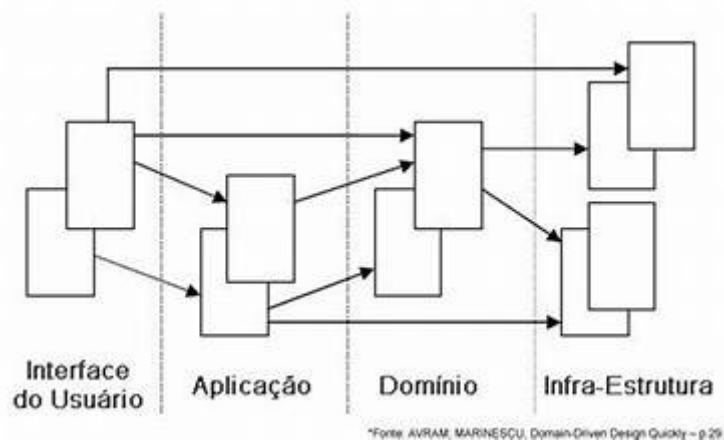


Figura 4- Arquitetura

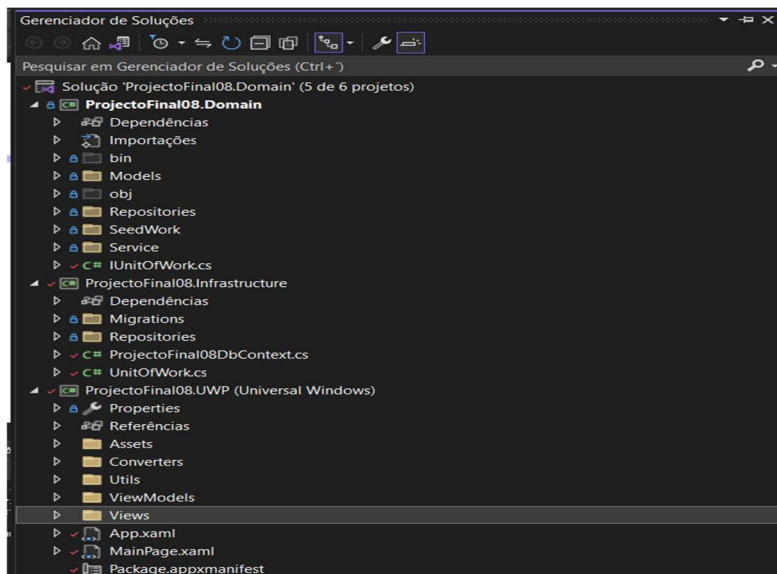


Figura 5- Organização do projeto

4.2.1 Camada de Domínio

Esta camada centraliza a lógica e as regras de negócio, os modelos dos domínios da aplicação, entre outras abstrações, bem como os modelos e repositórios:

- Modelos: é onde são definidas as entidades do sistema, representando os principais objetos do sistema, como o Usuário, Serviço, Requisição e etc...
- Repositórios: que contém as interfaces que definem as regras de acesso aos dados;
- Serviço: que tem por finalidade executar alguma lógica de acesso à base de dados que manipula as entidades definidas no domínio (Domain) e delegar algumas ações a serem tomadas para a camada de dados.
- SeedWork: que geralmente inclui todas as classes e interfaces que são utilizadas em todas as classes do domínio, como por exemplo a classe Entity que é a base das entidades do componente Models;
- IUnitOfWork.cs: é onde é definida a interface para o padrão Unit of Work, que nada mais é do que um padrão de projeto que gira em torno do gerenciamento eficaz de transações de banco de dados. Ele fornece um mecanismo centralizado para rastrear alterações, garantindo commits atômicos ou rollbacks, melhorando assim a manutenibilidade do código e promovendo uma separação mais limpa entre a lógica de negócio e as preocupações com o acesso aos dados.

4.2.2 Camada de infraestrutura

Esta camada é responsável pela implementação do banco de dados, persistência e acesso aos dados no banco de dados, portanto é nela que serão implementados os códigos para consultas, inserções e outras coisas que envolvam manipulação de dados.

Seus componentes são:

- Migration : Utilizado para gerenciar as migrações do banco de dados, que nada mais é do que aplicar ou reverter as alterações feitas no banco de dados conforme a evolução da aplicação. Com ele, é possível atualizar facilmente o esquema da base de dados caso seja necessário, sem precisar escrever scripts manuais ou se preocupar com a perda de dados;
- Repositórios: que nada mais são do que as implementações concretas das interfaces previamente definidas no repositório da camada de domínio;
- ProjectoFinalDbContext.cs: representa o contexto da base de dados, gere a sua ligação e mapeia as entidades do domínio para as tabelas;
- UnitOfWork.cs: tal como os “Repositórios”, é a implementação concreta da interface IUnitOfWork previamente definida na camada de Domínio.

4.2.3 Camada de apresentação

Esta camada é responsável pela implementação da interface do utilizador na plataforma, que permite ao utilizador interagir com o sistema.

Os seus componentes:

- Utils: que contém funções que ajudam a implementar esta camada.
- ViewModels: este componente contém a lógica de apresentação da plataforma que liga a IU à camada de negócio seguindo o padrão MVVM (Model- Biew-ViewModel).
- Converters: que contém os conversores que ajudam e facilitam a ligação de dados entre a IU e os ViewModels.
- Views: onde são definidas as páginas que compõem a interface gráfica da plataforma

- App.xaml e MainPage.xaml: são os ficheiros centrais da plataforma e definem a sua navegação inicial.

4.3 Base de dados

A seguir, será apresentado uma parte do código implementado referente a base de dados. Na criação da base de dados, foram definidas todas as tabelas e relações necessárias no desenvolvimento do projeto, de acordo com os requisitos identificados no capítulo 3. Foi utilizado o Entity Framework Core para a realização do mapeamento das entidades para a base de dados e as interações realizadas entre elas, no método OnModelCreating do DbContext, foram definidas as restrições para cada tabela ou entidade como chaves primárias, índices únicos, campos obrigatórios e limites de tamanho e etc...

Abaixo será apresentado em código a criação das seguintes tabelas: Cliente, Utilizadores e Laboratório, e também será apresentado em código algumas relações entre as tabelas.

4.3.1 Criação das Tabelas

- Tabela Cliente

```
mb.Entity<Cliente>().Property(c                                =>
c.Username).IsRequired().HasMaxLength(100);

mb.Entity<Cliente>().HasIndex(c => c.Username).IsUnique();

mb.Entity<Cliente>().Property(c                                =>
c.NomeCliente).IsRequired().HasMaxLength(100);

mb.Entity<Cliente>().HasIndex(c                                =>
c.NifCliente).IsUnique();

mb.Entity<Cliente>().Property(c                                =>
c.NifCliente).HasMaxLength(20);

mb.Entity<Cliente>().HasIndex(c                                =>
c.TelefoneCliente).IsUnique();

mb.Entity<Cliente>().Property(c                                =>
c.TelefoneCliente).HasMaxLength(20);
```

```

        mb.Entity<Cliente>().HasIndex(c =>
c.EmailCliente).IsUnique();

        mb.Entity<Cliente>().Property(c
c.EmailCliente).HasMaxLength(100);

        mb.Entity<Cliente>().Property(c
c.MoradaCliente).HasMaxLength(200);

        mb.Entity<Cliente>().Property(c=>
c.Sobrenome).HasMaxLength(200);

        mb.Entity<Cliente>().Property(c
c.Password).IsRequired().HasMaxLength(10);

        mb.Entity<Cliente>().Property(c
c.IsArquivado).IsRequired();

        mb.Entity<Cliente>().Property(c
c.IsBloqueado).IsRequired();

```

Aqui são definidos os campos da entidade Cliente sendo que 5 deles nomeadamente o Username, Nome do cliente (NomeCliente), Password e os booleanos IsArquivado e IsBloqueado, são declarados como obrigatórios ou seja não podem ser nulos pois os booleanos por estes servem para indicar o status do cliente na plataforma(se este está bloqueado ou arquivado), e os demais para identificar individualmente e de forma mais extensa o cliente e permitir o acesso a plataforma pelo cliente utilizador, é definido o tamanho máximo de caracteres para cada atributo o índice único para evitar duplicação indesejada de dados.

- Utilizadores (Funcionários da Instituição)

```

        mb.Entity<Utilizador>().HasIndex(c =>
c.UserName).IsUnique();

        mb.Entity<Utilizador>().Property(c =>
c.UserName).IsRequired().HasMaxLength(50);

        mb.Entity<Utilizador>().HasIndex(c =>
c.NomeUtilizador).IsUnique();

```

```

        mb.Entity<Utilizador>().Property(c =>
c.NomeUtilizador).IsRequired().HasMaxLength(100);

        mb.Entity<Utilizador>().HasIndex(c =>
c.NifUtilizador).IsUnique();

        mb.Entity<Utilizador>().Property(c =>
c.NifUtilizador).HasMaxLength(10);

        mb.Entity<Utilizador>().HasIndex(c =>
c.TelefoneUtilizador).IsUnique();

        mb.Entity<Utilizador>().Property(c =>
c.TelefoneUtilizador).HasMaxLength(20);

        mb.Entity<Utilizador>().HasIndex(c =>
c.EmailUtilizador).IsUnique();

        mb.Entity<Utilizador>().Property(c =>
c.EmailUtilizador).HasMaxLength(100);

        mb.Entity<Utilizador>().Property(c =>
c.MoradaUtilizador).HasMaxLength(200);

        mb.Entity<Utilizador>().Property(c =>
c.Sobrenome).HasMaxLength(200);

        mb.Entity<Utilizador>().Property(c =>
c.Password).IsRequired().HasMaxLength(10);

        mb.Entity<Utilizador>().Property(c =>
c.IsAdmin).IsRequired();

        mb.Entity<Utilizador>().Property(c =>
c.IsArquivado).IsRequired();

        mb.Entity<Utilizador>().Property(c =>
c.IsBloqueado).IsRequired();

```

Muito semelhante à do cliente pese embora a existência de alguns campos distinto como neste caso o atributo booleano IsAdmin que serve para indicar se o utilizador é o administrador da plataforma ou não.

- Tabela Laboratório

```
mb.Entity<Laboratorio>().HasIndex(c =>
c.NomeLaboratorio).IsUnique();

mb.Entity<Laboratorio>().Property(c=>
c.NomeLaboratorio).IsRequired().HasMaxLength(9);

mb.Entity<Laboratorio>().Property(c=>
c.DescricaoLaboratorio).IsRequired().HasMaxLength(256);

mb.Entity<Laboratorio>().HasIndex(c =>
c.EmailLaboratorio).IsUnique();

mb.Entity<Laboratorio>().Property(c=>
c.EmailLaboratorio).IsRequired().HasMaxLength(40);

mb.Entity<Laboratorio>().HasIndex(c =>
c.ContactoLaboratorio).IsUnique();

mb.Entity<Laboratorio>().Property(c=>
c.ContactoLaboratorio).IsRequired().HasMaxLength(10);

mb.Entity<Laboratorio>().Property(c =>
c.IsArquivado).IsRequired();
```

- Similar a implementação das tabelas anteriores, aqui também são definidos os campos obrigatórios, o campo IsArquivado que indica o status do Laboratório, e etc...

4.3.2 Relação entre as tabelas

Nesta parte é definida o relacionamento entre as entidades ou tabelas bem com as suas respetivas restrições para evitar perda de dados considerados importantes.

- Relação entre Pedido e Cliente

```
mb.Entity<Pedido>()
    .HasOne(a => a.Cliente)
```

```

.WithMany(p => p.Pedidos)

.HasForeignKey(a => a.ClienteId)

.OnDelete(DeleteBehavior.Restrict);

```

Aqui é dito como o Pedido está relacionado com o Cliente através da chave estrangeira (ClienteId), foi definido que um cliente pode ter vários pedidos (WithMany p=> p.pedidos), mas em contrapartida cada Pedido pertence a um único Cliente, também de definido o comportamento de exclusão, sendo neste caso optado por definir como Restrict, ou seja caso o cliente tenha um pedido associado a ele, o mesmo não pode ser excluído.

- Relação entre Utilizador e Laboratório

```

mb.Entity<Utilizador>()

.HasOne(a => a.Laboratorio)

.WithMany(s => s.Utilizadores)

.HasForeignKey(a => a.LaboratorioId)

.OnDelete(DeleteBehavior.Cascade)

```

Aqui similar ao caso anterior, um laboratório pode ter vários utilizadores (que neste caso são os funcionários da instituição registados na plataforma), mas o mesmo só podem está associado a um só Laboratório, diferente do caso acima e exclusão é em Cascata ou seja, na exclusão de um laboratório todos os utilizadores associados a eles serão excluídos.

- Relação entre pedido e Serviço por meio tabela relacional PedidoServico

Relação entre Pedido e PedidoServico

```

.HasForeignKey(a => a.Laboratorio

mb.Entity<PedidoServico>()

.HasOne(a => a.Pedido)

.WithMany(p => p.PedidoServicos)

.HasForeignKey(a => a.PedidoId)

.OnDelete(DeleteBehavior.Restrict);

```

Relação entre Servico e pedidoServico

```
mb.Entity<PedidoServico>()  
.HasOne(a => a.Servico)  
.WithMany(p => p.PedidoServicos)  
.HasForeignKey(a => a.ServicoId)  
.OnDelete(DeleteBehavior.Restrict);
```

Sendo este tipo de relação denominada de muitos para muitos em que um pedido pode ter vários serviços e de igual forma um serviço pode estar associado a vários Pedidos, foi necessária criação de uma tabela relacional denominada PedidoServico que fará a vinculação de um ou mais pedidos a um o mais serviço nomeadamente, ambos os relacionamentos das tabelas pedido -> pedidoServico, e serviço -> pedidoServico exclusão foi definida como Restrict (restrita) o que garante a integridade dos dados.

4.2.3 Implementação de métodos de manipulação de dados

Depois de criada as entidades é necessário implementar as operações essenciais para uma boa gestão dos dados na plataforma. Nesta etapa será descrita a implementação de algumas dessas operações, para isso utilizou-se também o Entity Framework Sqlite, que facilitou a implementação devido a sua abrangente biblioteca de ferramentas integradas para efeitos de manipulação de dados, os demais método não menos importantes muito pelo contrário, podem ser encontrados definidos em interfaces localizadas na paste “Repositories” na camada de domínio(ProjectoFinal08.domain), já as implementações destes podem ser encontrada nas classe da pasta “Repositories” na camada de infraestrutura.

As operações escolhidas para serem descritas são as CRUD, estas nada mais são que as 4 operações básicas que a plataforma deve ser capaz de realizar sendo essas o Create, Read, Update e Delete.

4.2.3.1 Operação Create(criação de entidade)

```
public T Create(T entity)  
{  
    T enti = _dbContext.Set<T>().Add(entity).Entity;  
    return enti;
```



```
}
```

Esta operação realiza a criação de uma nova entidade no contexto da base de dados, o parâmetro T representa a entidade que será adicionada a base de dados (exemplo um novo Cliente), é feita a sua adição, e por fim retornado a mesma.

4.2.3.2 Read (Leitura)

```
public async Task<T> FindByIdAsync(int id){  
    return await _dbContext.Set<T>().FindAsync(id);  
}
```

Esta operação realiza a busca de uma entidade baseada no seu identificador (Id) correspondente, e retorna a entidade encontrada.

4.3.3 4.2.3.3 Update(atualização)

```
public T Update(T entity){  
    _dbContext.Entry(entity).State = EntityState.Modified;  
    T enti = _dbContext.Set<T>().Update(entity).Entity;  
    return enti;  
}
```

Esta operação realiza a modificação e atualização de uma entidade (T) correspondente, e retorna a entidade modificada.

4.3.4 4.2.3.4 Delete(Exclusão)

```
public T Delete(T entity){  
    T enti = _dbContext.Set<T>().Remove(entity).Entity;  
    return enti;  
}
```

Desta forma é realizada operação de exclusão, ou seja, remover de uma determinada entidade(T)no contexto da base de dados.

4.4 Implementação da plataforma

Após serem definidas as entidades, e ser feita implementação da base de dados bem como as operações a serem efetuadas nela, este tópico apresenta a logica e o desenvolvimento da interface gráfica da plataforma. Será descrita a implementação dos requisitos anteriormente citados no capítulo 3 tais como, permitir o registo, a autenticação dos utilizadores na plataforma, permitir ao utilizador cliente depois de estar logado efetuar um ou mais pedidos bem como geri-los, permitir a cada utilizador editar o seu perfil, efetuar o regaste da sua senha de acesso, permitir ao administrador gerir os dados apresentados na aplicação como

por exemplo arquivar ou bloquear um determinado utilizador, aceitar ou recusar um pedido feito pelo cliente, entre outras operações.

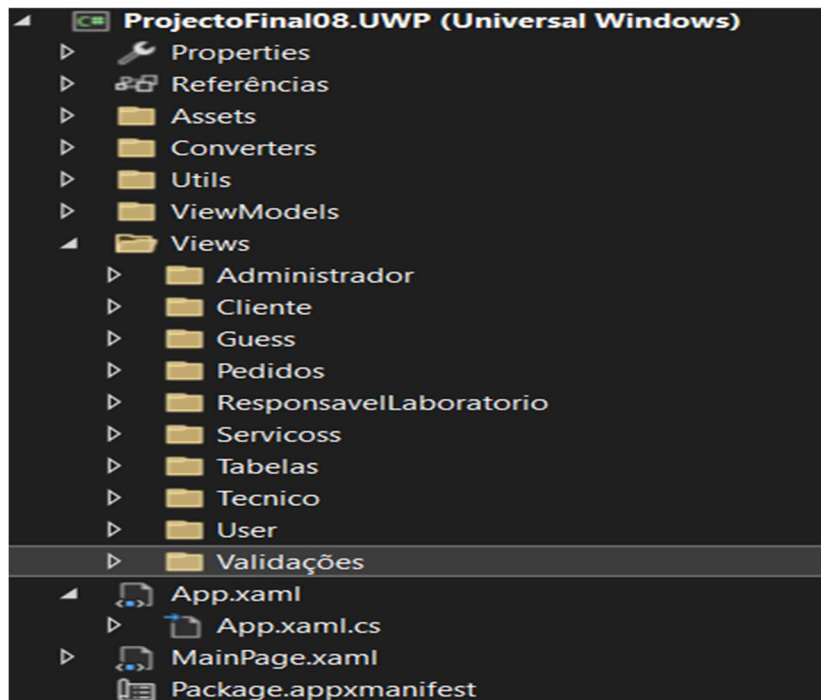


Figura 6: Estrutura do projeto uwp

Como dito anteriormente na camada de apresentação o projeto está estruturado da forma mostrada na imagem abaixo. Foi utilizado o padrão de design MVVM(Model-View-ViewModel) para promover a separação de responsabilidades.

- ViewModels : Cada ViewModel criado é responsável por gerenciar os dados e comportamentos necessários para cada View (tela ou página), por exemplo o ServicoViewModel é responsável por lidar com todas das operações, relacionadas aos Serviços desde a criação de um, além de validar dados antes de envia-los para a base da dados.
- View : As views todas foram implementadas em xaml para definir a interface visual da aplicação. A interação entre esta e o ViewModel é feita através de binding, o que permite a modificação dos dados seja automaticamente refletida na interface.

4.4.1 Registo do Cliente

O registo do cliente está dividido em duas etapas. A primeira consiste no cliente preencher o formulário aonde são pedidos alguns dados relevantes são como nome, sobrenome, Nif, telefone, email e etc...., depois este serão analisados se cumprem com a regras por exemplo o numero de telefone tem de ter 9 dígitos, Nif também, a password deve ter pelo menos 8 caracteres em que pelo menos um é uma letra maiúscula, um número, um caractere especial e não pode repetir qualquer caractere mais de 3 vezes e .etc. A segunda etapa é a aceitação ou não do registo do cliente por parte do administrador, este pode ou não ser validado caso por exemplo o cliente coloque algum dado que possa ser considerado impróprio no formulário de registo, mas caso o registo seja aceito o mesmo está à-vontade para aceder e disfrutar de tudo o que a plataforma tem a lhe oferecer. E o mesmo receberá um email a lhe comunicar que já pode aceder a plataforma com as suas credenciais, para poder efetuar pedidos, visualizar os serviços disponíveis e etc...

A função de registo do cliente está implementada da seguinte forma:

```
private async void RegistoButton_Click(object sender,
RoutedEventArgs args)
{
    string response = "Falha no Registo por favor
verique se os dados inseridos são validos ";

    if (UsernameText.Text.Equals("") ||
NameText.Text.Equals("") || EmailText.Text.Equals("") ||
    TelefoneText.Text.Equals("") ||
AsbLocal.Text.Equals("") || LastNameText.Text.Equals("") ||
    MoradaText.Text.Equals("") ||
PasswordText.Password.Equals("") ||
RPasswordText.Password.Equals(""))
    {
        await new
Windows.UI.Popups.MessageDialog("Insira todos os dados
necessários").ShowAsync();
    }
    else if (PasswordText.Password !=
RPasswordText.Password)
    {
        await new
Windows.UI.Popups.MessageDialog("As passwords não
coinsidem").ShowAsync();
    }
    else if (!Regex.IsMatch(EmailText.Text,
@"^[^@\s]+@[^@\s]+\.[^@\s]+$"))
    {
        await new
Windows.UI.Popups.MessageDialog("Email
invalido").ShowAsync();
    }
}
```

```

        else if (!Regex.IsMatch(TelefoneText.Text,
@"^\d{9}$"))
        {
            await new
Windows.UI.Popups.MessageDialog("O número de telefone não é
válido").ShowAsync();
        }
        else if (!Regex.IsMatch(NifText.Text,
@"^\d{9}$"))
        {
            await new
Windows.UI.Popups.MessageDialog("O número de contribuinte não
é válido").ShowAsync();
        }
        else if
(!IsValidPassword(PasswordText.Password))
        {
            await new MessageDialog("Password
inválida. A senha deve ter pelo menos 8 caracteres em que pelo
menos um é uma letra maiúscula, um número, um caractere
especial e não pode repetir qualquer caractere mais de 3
vezes.").ShowAsync();
        }
        else
        {
            if( await ClienteViewModel.RegistarAsync())
            {
                Frame.Navigate(typeof(GuessPage));
                await new MessageDialog("Registo
efetuado com sucesso, por favor aguarde a ativação da sua
conta de utilizador").ShowAsync();
            }
            else
                await new
Windows.UI.Popups.MessageDialog(response).ShowAsync();
        }
    }
}

```

4.4.2 Autenticação (Login)

Após o registo de Cliente ser aceite ou o utilizador Funcionário (responsável de laboratório ou técnico) ser criado, ao mesmo em posse dos seus dados de entrada validos, podem aceder a sua determinada área na plataforma, para execução desta operação é usada a função criada no ViewModel dos utilizadores de nome “DoLoginAsync(usernameEmail, password)” esta verifica se dados inseridos pelo utilizador são validos ou seja se encontram definidos na base de dados, caso as credenciais estejam erradas é devolvido uma mensagem de erro.

4.4.3 Recuperação de Senha de acesso

Esta funcionalidade foi implementada com base num sistema de envio de mail, em que o utilizador fornece o seu email registado na plataforma e caso o mesmo seja válido, é enviado um email com o código de recuperação da conta e ao voltar para a aplicação a mesma mostrará a pagina aonde o mesmo poderá inserir este código e a sua nova password, e pedido para esta ser inserida 2 vezes para prevenir erros de digitação e confirmação se o mesmo quer realmente esta senha de acesso para a plataforma.

```
internal async void AlterarPasswordCliente(Cliente e,
string newPass)
{
    using (var uow = new UnitOfWork())
    {
        var cliente = e.ID;

        if (cliente != 0)
        {
            await
uow.ClienteRepository.AlterarPassword(cliente, newPass);
            uow.SaveAsync();
        }
        else
        {
            Console.WriteLine("Cliente não
encontrado");
        }
    }
}
```

4.4.4 Gerenciar pedidos

Esta funcionalidade é realizada pelos clientes, estes podem criar alterar ou mesmo eliminar o pedido.

A eliminação de um pedido depende do seu estado atual exemplo caso o estado diferente do efetuado ou alterado o mesmo não pode ser eliminado, todas estas operações podem ser observadas no ficheiro ViewModel dos Pedidos. Cada pedido é armazenado no banco de dados e pode ser atualizado ou cancelado antes de ser processado.

4.4.5 Gerenciar Utilizadores (Clientes ou Funcionário)

Esta funcionalidade é realizada pelos Administrador e também pelo responsável de laboratório caso o utilizador seja o Técnico, estes podem criar, alterar, ou mesmo arquivar ou mesmo eliminar depois de arquivado o utilizador.

A eliminação de um utilizador depende do seu estado de status atual por exemplo caso o mesmo não esteja bloqueado ou arquivado, ou se for um cliente que tenha um pedido ainda por terminar, o mesmo não pode ser eliminado, todas estas operações podem ser observadas no ficheiro ViewModel dos Utilizadoar(funcionários) e do Cliente respetivamente.

4.4.6 Gerenciar serviços

Esta funcionalidade é realizada pelos Administrador, estes podem criar, alterar, Arquivar entre outras operações, todas estas podem ser observadas no ficheiro ViewModel dos Serviços. Cada pedido é armazenado no banco de dados e pode ser atualizado ou cancelado antes de ser processado.

4.4.7 Gerenciar Orçamentos

Esta funcionalidade é realizada pelos Técnico, estes entra outras operações podem criar, alterar, ou mesmo eliminar um orçamento dependendo do estado do pedido. Todas estas operações podem ser observadas no ficheiro ViewModel dos Pedidos. Cada pedido é armazenado no banco de dados e pode ser atualizado ou cancelado antes de ser processado.

4.4.8 Envio de Email

Foi implementado na plataforma um função que permite enviar e-mail automático, caso sejam efetuada as seguintes operações:

- Criação de contas;
- Recuperação de senha de acesso,
- Criação de pedido pelo cliente
- Alteração do Pedido,
- Elaboração do orçamento;
- O estado do pedido,
- Alteração do status do Utilizador

4.4.9 Download de ficheiro

Na plataforma é possível efetuar o download dos ficheiros das tabelas apresentadas. Abaixo se encontra o exemplo dessa operação para a tabela de pedidos:

```
private async void BtnExport_Click(object sender,
RoutedEventArgs e)
```

```

        {
            ExcelPackage.LicenseContext =
LicenseContext.NonCommercial;
            ExcelPackage package = new ExcelPackage();

            ExcelWorksheet worksheet =
package.Workbook.Worksheets.Add("Dados");

            for (int i = 0; i < dataGrid1.Columns.Count;
i++)
            {
                worksheet.Cells[1, i + 1].Value =
dataGrid1.Columns[i].Header;
            }

            var itemsSource = dataGrid1.ItemsSource as
IEnumerable<Pedido>;
            int row = 2;
            foreach (var item in itemsSource)
            {
                worksheet.Cells[row, 1].Value = item.ID;
                worksheet.Cells[row, 2].Value =
item.ClienteId;
                worksheet.Cells[row, 3].Value =
item.Cliente?.NomeCliente ?? "";
                worksheet.Cells[row, 4].Value =
item.DataPedido;
                worksheet.Cells[row, 5].Value =
item.Estado;
                row++;
            }

            FileSavePicker savePicker = new
FileSavePicker();
            savePicker.SuggestedStartLocation =
PickerLocationId.DocumentsLibrary;
            savePicker.FileTypeChoices.Add("Excel
Workbook", new List<string>() { ".xlsx" });
            savePicker.SuggestedFileName = "Dados";

            StorageFile file = await
savePicker.PickSaveFileAsync();
            if (file != null)
            {
                Stream stream = await
file.OpenStreamForWriteAsync();
                package.SaveAs(stream);
                stream.Close();
            }
        }
    }

```

5

Interfaces

O objetivo deste capítulo, é apresentar e explicar de forma clara toda a Interface desenvolvida neste, e o funcionamento projeto. São apresentadas imagens e explicações de cada página que constituem o projeto.

Página Inicial



Figura 7- página inicial

Serviços

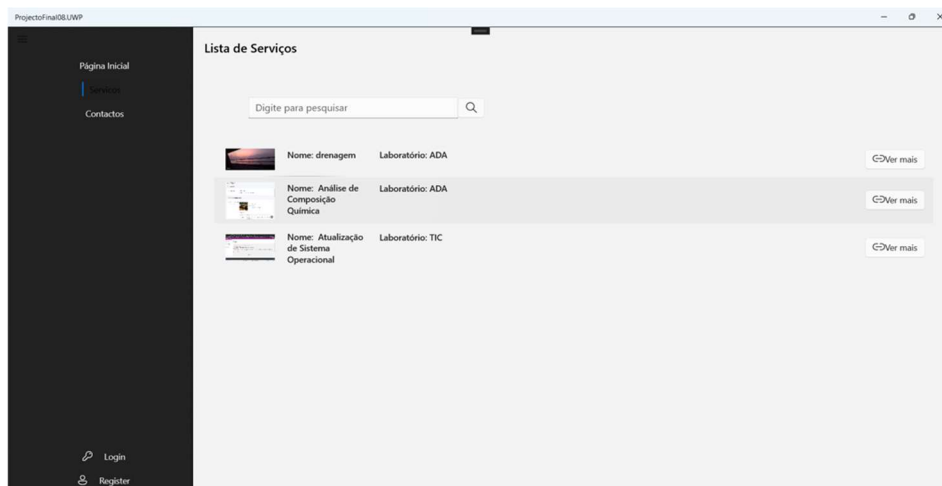


Figura 80 - Serviços

Aqui é onde os futuros cliente visitantes da plataforma faz a observa os serviços prestados pela Instituição, podendo arquivar serviços, editar e adicionar novos serviços

Registrar

Se o utilizador não estiver registado, terá um botão para se registar.

Diferente dos utilizadores de que fazem parte da Instituição (responsável de laboratório e técnico) os clientes são permitidos a criarem as suas próprias contas de acesso a plataforma a partir do formulário abaixo:

ctoFinal08.UWP

ipb INSTITUTO POLITÉCNICO DE BRAGANÇA
Escola Superior de Tecnologia e Gestão

Registo

Adicionar foto de perfil

Username
User Name

Nome do Cliente Sobrenome
Nome Sobre nome

Número de Contribuinte
0

Email do Cliente
Email

Telefone
Telefone

Morada Cidade
Morada Q

Figura 9- Registo

Password
Password

Confirmar Password

Registar Cancelar

[Já Possui uma Conta? Login](#)

Login

Após o registo na plataforma os utilizadores já poderão aceder a plataforma no com as suas credências.

No caso dos clientes com os dados introduzidos no momento do registo e no caso dos funcionários (responsável de laboratório ou técnico) com os dados que lhes são atribuídos pelo administrador.

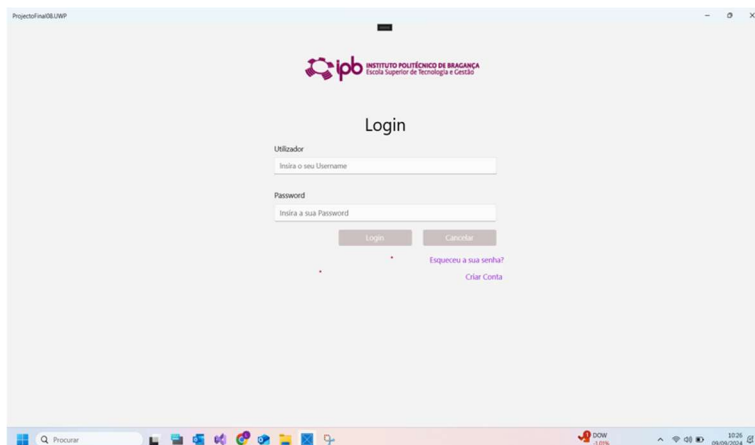


Figura 10- Login

Recuperação da Senha

O Utilizador efetua este requisito clicando a botão “Esqueceu a sua senha” que o enviará a página abaixo aonde o mesmo terá de inserir o seu email.

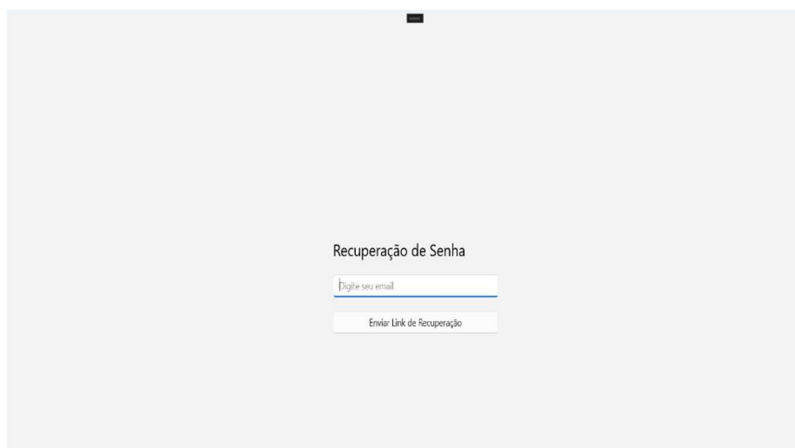


Figura 11- Recuperação de Senha

Após o envio do pedido, será enviado ao utilizador um e-mail para o endereço introduzido com um código de recuperação e será efetuada uma alteração na plataforma.

Em seguida aparece uma tela para inserir o código enviada e a nova palavra-passe.



4.5 Área do Utilizador

Os utilizadores Admin, Responsável de laboratório e Técnico de laboratório ao efetuarem login são enviados para a página de notificações para que já de primeira possam observar os pedido de validações pendentes do pedido efetuado pelo cliente ou do orçamento efetuado(admin responsável de labora) ou de reavaliação do relatório (Técnico).

1.1.12 Admin

No caso de o utilizador admin efetuar login, é apresentado um botão para fazer logout e as seguintes opções:

- Serviços
- Laboratório
- Localidade
- Dados Arquivados
- Tipo De Utilizadores
- Orçamento
- Pedidos
- Perfil
- Notificações

Laboratórios

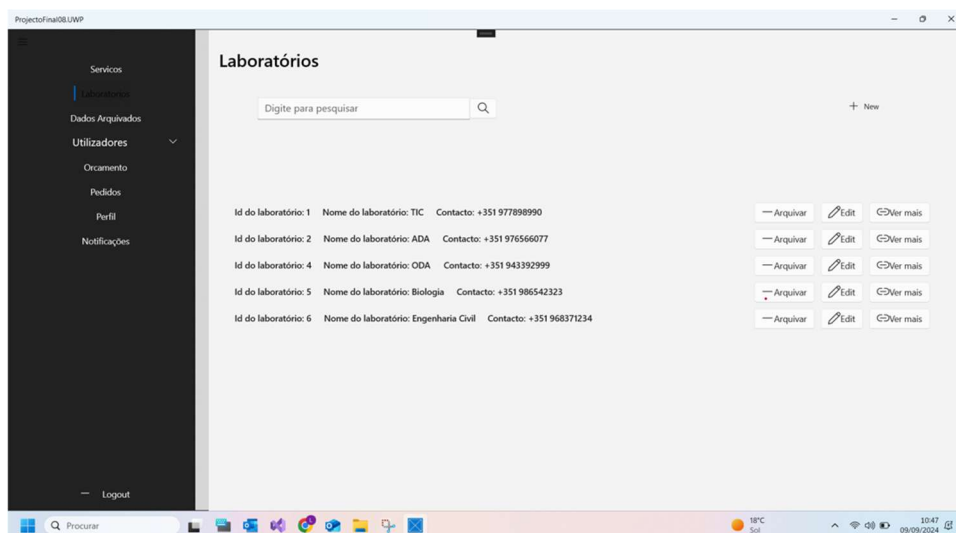


Figura 13- Laboratórios

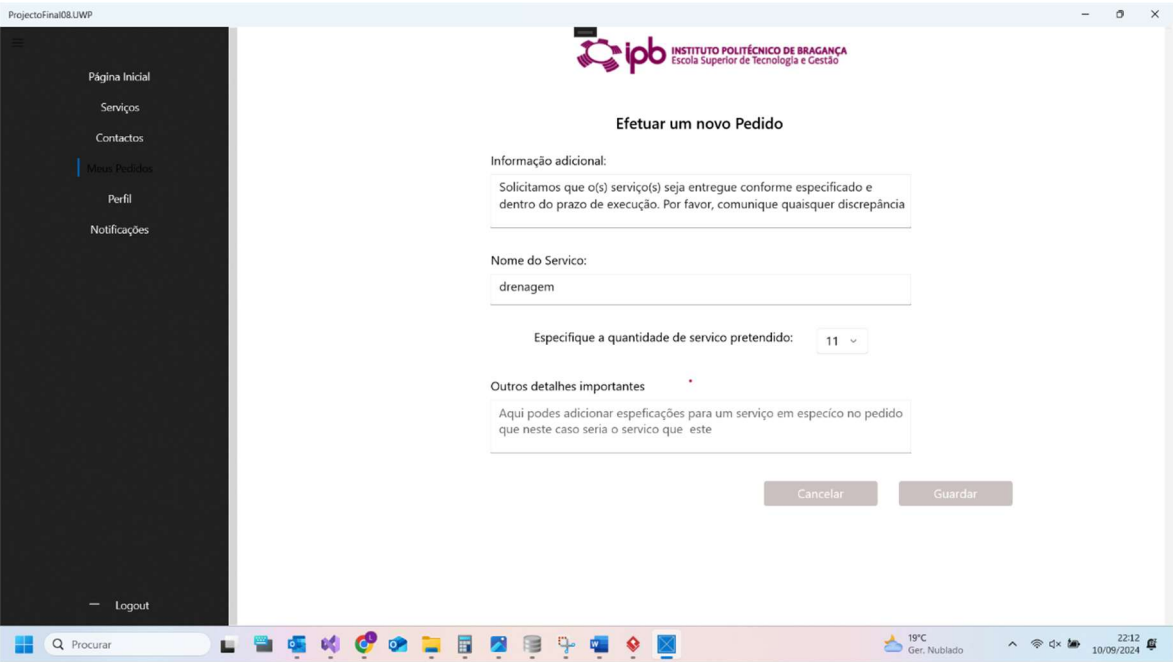
Aqui são listados os laboratórios com as seguintes ações:

Arquivar: o administrador pode arquivar os laboratórios que já não prestam mais serviços.

Editar: aqui o administrador pode fazer as alterações necessárias, ou seja, alteras as informações dos laboratórios.

New: aqui o administrador pode adicionar novos laboratórios.

Pedidos



Aqui são representadas a página de elaboração dos pedidos com todas as informações desde o ID do cliente até o estado dos pedidos.

Tipo de Utilizador

The screenshot shows a web application interface for managing clients. On the left is a dark sidebar with navigation links: 'Serviços', 'Laboratórios', 'Todos Registos', 'Utilizadores' (highlighted), 'Orçamento', 'Pedidos', 'Perfil', and 'Notificações'. At the bottom of the sidebar is a 'Logout' link. The main content area has a header with the title 'Tabela de Clientes'. Below the header is a search bar with the placeholder text 'Digite para pesquisar' and a magnifying glass icon. To the right of the search bar is a '+ New' button. Below the search bar is a table with the following columns: ID, Username, Nome, Apellido, NIF, Email, Telefone, Morada, Cidade, situação da conta, and A. The table contains 11 rows of client data.

ID	Username	Nome	Apellido	NIF	Email	Telefone	Morada	Cidade	situação da conta	A
1	lili	Liliana	Paquete	297563065	lilianapaquete@outlook.com	967352718	Rua fernando lima	Setúbal	False	1
2	gui	gui	Veiga	987659998	gsocial1009@gmail.com	983754635	Sacavem	Bragança	False	1
4	Emisa12	Emilena	Sá	298765439	emilenasa98@gmail.com	987654573	Avenida Sa Carneiro	Lisboa	False	1
5	kelly15	kelly	Fernandes	317654636	kelly@gmail.com	987654124	Avenida Sa Carneiro	Lisboa	False	1
6	Cv14	Carlos	Veiga	287563278	carlosveiga77@gmail.com	966472534	Rua Pero Escobar	Lisboa	False	1
8	ted	ted	Sousa	679554554	da124lopes@gmail.com	286599999	Rua visconde da bouca	Lisboa	False	1
11	rita	rita	Panuarte	987654444	rita@panuarte@gmail.com	298765455	Avenida cinto Maria	Lisboa	False	1

Figura 14 -Tipo de Utilizadores (Clientes)

Nesta página o administrador consegue ver a lista dos clientes podendo arquivar os clientes, alterar os dados dos clientes e adicionar novos clientes

ID	Username	Nome	NIF	Email	Telefone	Laboratorio	situação da conta	Morada	Cargo
2	re1	re1	298760987	dailypaquete@gmail.com	929876549	ADA	False	avenida S.tomé o principe n 67	Resp
3	tec1	tec1	188887657	da124lopes@gmail.com	934529876	ADA	False	Via Santa Catarina n 20	Téc
4	JR24	Jose	319876543	re2@gmail.com	987298765	TIC	False	Agualva-Cacém	Resp
5	Valopes	Vanessa Lopes	280987654	Vanessalop@gmail.com	982987654	TIC	False	Via Santa Catarina n 10	Téc
6	KUK	KUK	296779929	kuk@gmail.com	973298765	ADA	False	santa apolonia n 24	Téc

Figura 15 -Tipo de Utilizadores (Técnicos e Responsáveis de Laboratórios)

Nesta página o administrador consegue ver a lista dos funcionários (Técnicos e Responsáveis de Laboratórios) podendo arquivar os funcionários, alterar os dados dos funcionários e adicionar novos funcionários.

1.1.13 Cliente

No caso de o utilizador cliente efetuar login, é apresentado um botão para o utilizador fazer logout e as seguintes opções:

- Página inicial
- Serviços
- Contacto
- Meus Pedidos
- Perfil
- Notificações

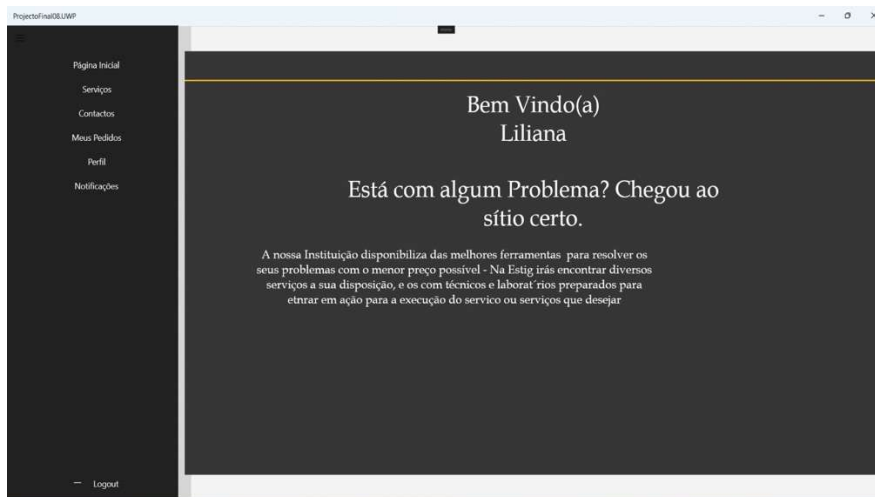


Figura 16 -Página Inicial Cliente

Pedidos

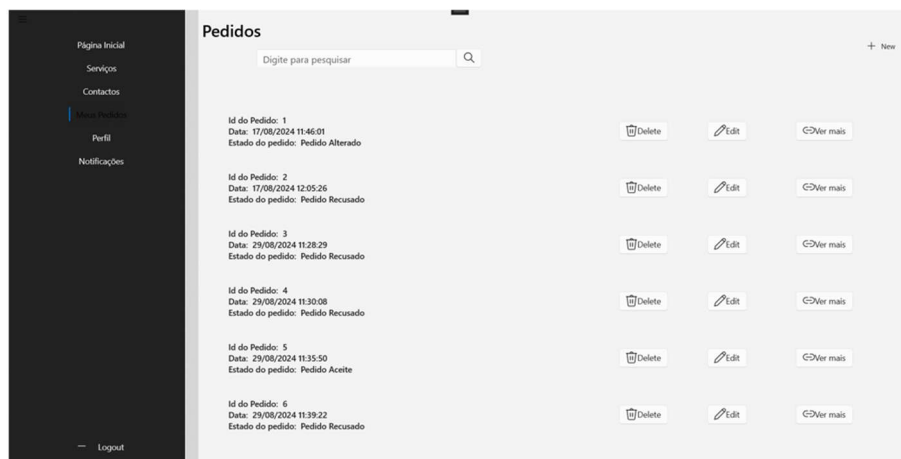


Figura 17 - Meus Pedidos (Cliente)

Nesta página os clientes encontram os pedidos feitos com todo o detalhe dos pedidos.

Tem a opção de arquivar o pedido editar pedidos que ainda não foram concluídos e pedir um novo orçamento.

1.1.14 Responsável de Laboratório

No caso de o utilizador responsável de laboratório efetuar login, é apresentado um botão para o utilizador fazer logout e as seguintes opções:

- Serviços
- Orçamento

- Pedidos
- Perfil
- Notificações

1.1.15 Técnico Laboratório

No caso de o utilizador técnico de laboratório efetuar login, é apresentado um botão para o utilizador fazer logout e as seguintes opções:

- Serviços
- Lista de Pedidos
- Orçamento
- Perfil
- Notificações

O técnico de Laboratório é quem elabora os orçamentos, em baixo será apresentado uma página onde isto é feito.

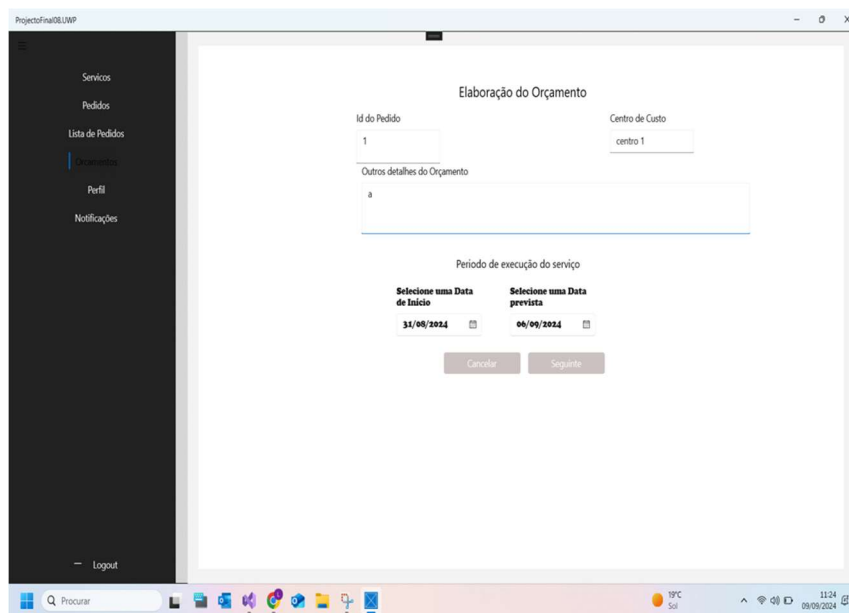


Figura 18 - Página Técnico de Laboratório (Elaboração de orçamento)

Pedidos

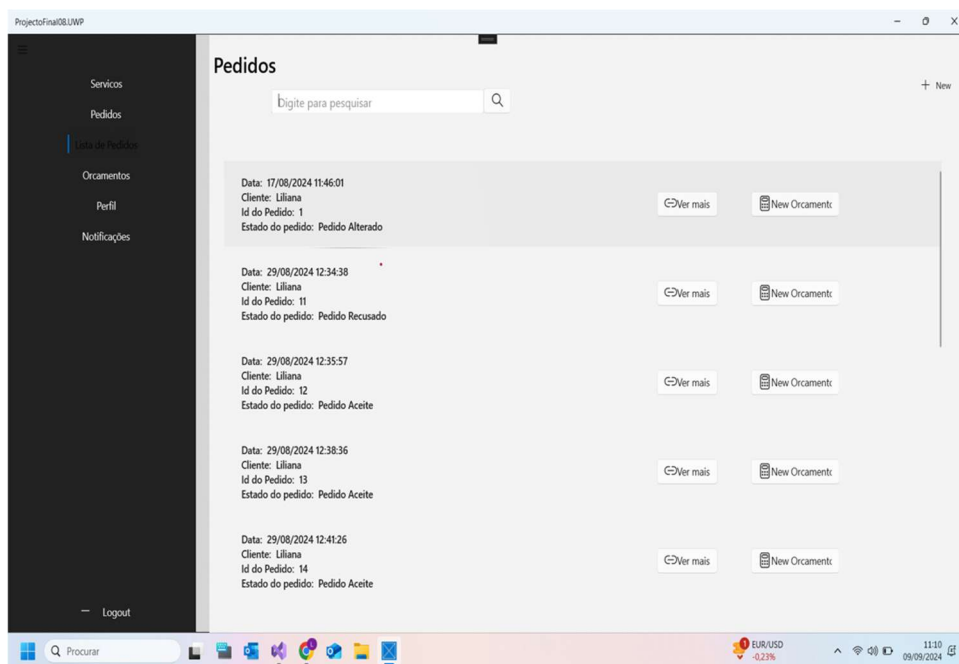


Figura 19 - Pedidos (Técnico de Laboratório)

Aqui é onde o técnico de laboratório vê a lista dos pedidos a serem realizados.

5

Conclusões

Numa perspetiva geral, o objetivo deste projeto foi o desenvolvimento de uma plataforma web capaz de otimizar a gestão e requisição dos serviços de apoio à comunidade prestados pela ESTIG. Esta plataforma pretende digitalizar e automatizar processos que anteriormente eram efetuados manualmente, minimizando o falhas humano e facilitando a comunicação entre os clientes e a instituição.

Durante o desenvolvimento deste projeto, foram estudadas as melhores práticas de gestão de serviços e as tecnologias mais adequadas para a construção de uma solução eficiente. Uma pesquisa aprofundada e uma implementação cuidada resultaram numa plataforma que melhora a qualidade dos serviços prestados pela ESTIG e a comunicação com os clientes.

Os resultados obtidos são animadores, mas ainda há aspetos a melhorar, como a interface.

Mas de todo é uma plataforma web bem estruturada e funcional, que demonstra a sua competência técnica. Este projeto representa muito no crescimento académico dos envolvidos, demonstrando a capacidade de enfrentar desafios.

Bibliografia

- [1] C#: Definição. Disponível em <https://www.infoescola.com/informatica/c-sharp/>
- [2] .NET: Definição. Disponível em <https://www.primeit.pt/net-tudo-o-que-precisas-saber>
- [3] Vantagens de C#: https://awari.com.br/c-e-a-linguagem-de-programacao-ideal-para-desenvolvedores-front-end-ou-back-end-2/?utm_source=blog&utm_campaign=projeto+blog&utm_medium=C#%20É%20A%20Linguagem%20De%20Programação%20Ideal%20Para%20Desenvolvedores%20Front-End%20Ou%20Back-End
- [4] O que é Visual Studio: <https://softtrader.pt/visual-studio/>
- [5] Vantagens de Visual Studio: <https://www.dio.me/articles/vantagens-e-desvantagens-do-uso-do-visual-studio-para-programar-em-c-net>
- [6] O que é UWP: https://pt.wikipedia.org/wiki/Plataforma_Universal_do_Windows
- [7] Vantagens de UWP: <https://learn.microsoft.com/pt-br/windows/uwp/get-started/universal-application-platform-guide>
- [8] SQLite: Definição, Vantagens Disponível em <https://coodesh.com/blog/dicionario/o-que-e-sqlite/>
- [9] SQLWorkbench: Definição, Vantagens Disponível em <https://melhorhospedagemdesite.com.br/glossario/o-que-e-mysql-workbench/>
- [10] Astah UML: Definição, Vantagens Disponível em <http://codilon.qlix.com.br/bd/bdaula09.pdf>